Linnet

Half-edge graphs and subgraph-first algorithms

1 Overview

Linnet is a graph library for tensor networks, Feynman diagrams, and other workflows where the boundary of a subgraph matters. Its central representation is a half-edge graph. An ordinary internal edge is a pair of half-edges, while an external edge has one dangling half-edge. A node owns incident half-edges, and a subgraph is a selection of half-edges rather than a detached copy of the graph.

> **Why half-edges**
> Cutting an internal edge can turn its two halves into boundary edges without changing the degree of either endpoint. This makes operations such as extraction, excision, contraction, sewing, traversal, and cut analysis natural for physics graphs. Linnet algorithms therefore accept subgraph views even when the caller wants to operate on the full graph.

1.1 A working model

A Linnet workflow has four parts:

- construct a graph with `HedgeGraphBuilder`, or parse a DOT graph;
- retain the typed identifiers returned for half-edges, nodes, and edges;
- describe the region of interest with one of the subgraph representations;
- run traversal or mutation operations against that subgraph and the graph's involution.

The involution is the pairing map for half-edges. A paired entry represents an internal edge; an identity entry represents a dangling edge and records its flow. Edge orientation and flow are related concepts, but they are not interchangeable. Consult the API reference for the exact variants and conversion rules in your Linnet release.

> **Indexes belong to one graph**
> `Hedge`, `NodeIndex`, and `EdgeIndex` are compact typed indexes, not durable object IDs. Graph edits can change storage and mappings. Keep the result returned by an operation, and do not apply an index or subgraph bitset to an unrelated graph just because its numeric values happen to fit.

1.2 Construction and interchange

The Rust package can be added directly from crates.io:

```
[dependencies]
linnet = "0.17"
```

Programmatic construction uses `HedgeGraphBuilder`. DOT support provides a convenient human-readable interchange and debugging format; it can carry global, node, edge, and half-edge attributes. Treat DOT as a representation boundary, not as a substitute for the typed graph invariants. Validate or handle parser errors before running algorithms.

The `drawing` feature adds graph drawing and layout support. Layout is separate from graph identity: coordinates and rendering attributes may change without changing the topology.

1.3 Related crates

One graph layer, several consumers
Spenso uses Linnet for tensor-network connectivity and adds tensor contraction semantics. GammaLoop uses the graph model in collider workflows and adds process generation, integration, and persistent state.

The Linnet README provides a compact crate summary; the Rust API reference covers individual types and methods.

2 Tutorial

In this tutorial, you will construct a half-edge graph in Rust, validate its invariants, find its one independent cycle, and print DOT. The builder returns Linnet's typed node and half-edge identifiers directly, so you can see where graph identity enters the workflow.

2.1 Prerequisites

Create a Rust binary project and add the current Linnet release:

```
cargo new linnet-first-graph
cd linnet-first-graph
cargo add linnet@0.17
```

No drawing feature or external layout program is needed to construct the graph and emit DOT.

2.2 Build a graph with a boundary

Replace `src/main.rs` with the following program:

```rust
use linnet::half_edge::{
    builder::HedgeGraphBuilder,
    involution::Flow,
    HedgeGraph,
};

fn main() {
    let mut builder = HedgeGraphBuilder::new();
    let a = builder.add_node("a");
    let b = builder.add_node("b");
    let c = builder.add_node("c");

    builder.add_edge(a, b, "ab", false);
    builder.add_edge(b, c, "bc", false);
    builder.add_edge(c, a, "ca", false);
    builder.add_external_edge(a, "incoming", false, Flow::Sink);
    builder.add_external_edge(c, "outgoing", false, Flow::Source);

    let graph: HedgeGraph<&str, &str> = builder.build();
    graph.check().expect("the half-edge involution is valid");

    let (basis, _) = graph.cycle_basis();
    assert_eq!(basis.len(), 1);
    println!("{}", graph.base_dot());
}
```

Run it with `cargo run`. Success means the assertion passes and stdout contains a DOT `digraph` with nodes `a`, `b`, and `c`, three paired internal edges, and two dangling boundary edges. The external edges do not add a cycle, so the triangle's cycle-basis rank remains one.

> **Keep the returned indexes with their graph**
> `a`, `b`, and `c` are compact `NodeIndex` values owned by this graph construction. Do not serialize their numeric representation as a durable identity or reuse them against another graph. Mutating operations return the mappings needed to relate old and new storage.

2.3 Make the first useful variation

Add a fourth internal edge between `a` and `b`, run the program again, and change the assertion to `basis.len() == 2`. This demonstrates the key distinction: parallel paired half-edges add an independent cycle, while dangling identity half-edges describe the graph boundary.

From here, use a subgraph view to run traversal, cuts, or contraction against only the region you intend to modify. Enable Linnet's `drawing` feature only when you need layout rather than plain DOT interchange.

2.4 Troubleshooting and next steps

- A type-inference error around `build()` means Rust cannot choose node storage or data types; keep the explicit `HedgeGraph<&str, &str>` annotation from the example.
- A failed `check()` points to an involution or storage invariant. Validate immediately after parsing or construction, before applying graph algorithms.
- If a traversal includes an unexpected dangling edge, check the selected subgraph and the `Flow` assigned to each external edge.
- Continue with the subgraph-and-algorithms manual, then consult the Rust API reference for the exact return mappings of mutating operations and additional builder methods.

3 Subgraphs, traversal, cuts, and drawing

Linnet algorithms operate on a graph together with an explicit subgraph view. This lets one algorithm work on the full graph, a selected region, or the boundary created by cutting paired half-edges. Choose the subgraph representation by operation: bit-backed sets are efficient for repeated set algebra, while borrowed or mapped views avoid copying when identity must remain attached to the owning graph.

3.1 Traversal and connectivity

A traversal needs a starting half-edge or node, the active subgraph, and a policy describing which adjacent half-edges may be crossed. Results retain typed indexes into the original graph. Breadth-first traversal is useful for distance layers and connectivity; depth-first traversal is useful for trees, back edges, and decompositions. A traversal tree is a derived view, not a new graph, so mutations must be coordinated with the indexes it contains.

Connected components, bridges, articulation structure, and biconnected regions all depend on the chosen subgraph. Run them on the full graph only when external half-edges and excluded nodes should participate in the answer.

3.2 Cycles and cuts

`cycle_basis` returns an independent basis and rank information. `all_cycles` expands that basis and can grow exponentially; prefer a basis when assigning loop momenta or computing a loop count. Symmetric differences combine cycle bitsets without inventing new graph identities.

Cut enumeration separates required left and right node sets and returns the left region, cut content, and right region. A cut edge is represented through its half-edges, preserving which side owns each endpoint. Validate

direction/flow after enumeration when a physical Cutkosky or tensor-network interpretation depends on orientation.

> **Complexity is part of the API choice**
> Enumerating every cycle or every admissible cut is inherently combinatorial. Filter the subgraph and endpoint sets first, and use basis or connectivity operations when enumeration is not required by the caller.

3.3 Mutation boundaries

Extraction copies a selected region, excision separates a graph along its boundary, sewing joins compatible dangling half-edges, and contraction identifies structure. These operations return mappings or replacement indexes where needed. Keep those results: a numeric `Hedge` or `NodeIndex` from before a storage-changing operation is not automatically a valid index into the resulting graph.

3.4 DOT and drawing

DOT parsing and emission are interchange/debugging boundaries. Attributes can carry graph, node, edge, and half-edge data, but the typed Rust invariants are reconstructed by the parser; handle parser errors before running algorithms. The `drawing` feature adds layout and rendering. Layout coordinates are presentation data and may vary without changing topology, subgraph membership, or edge flow.

> **Tensor contraction uses Spenso**
> Linnet determines connectivity and graph transformations. Tensor compatibility, contraction cost, and execution belong to Spenso even when a Spenso network delegates its graph algorithms to Linnet.

4 Rust and Python APIs

4.1 Rust package

The `linnet` package is organized into these principal areas:

- `half_edge` provides `HedgeGraph`, builders, the involution and edge data, subgraph types, traversal trees, and graph algorithms;
- `parser` provides DOT parsing and DOT-backed graph data;
- `drawing` provides rendering data and layout helpers;
- `permutation`, `tree`, and `union_find` provide supporting algorithms and stores.

The generic graph types separate edge, vertex, and half-edge payloads from the node-storage implementation. Start with the curated Rust API, then follow its Rustdoc links for complete generic bounds and method signatures; choose a concrete storage type according to the workflow and mutation behavior you need.

Cargo features enable optional capabilities:

- `serde` adds serialization traits;
- `bincode` adds binary encoding support;
- `drawing` adds the drawing stack;
- `symbolica` adds Symbolica interoperability.

An item shown in an all-features API reference may not be available in a default build. Check its required feature and your `Cargo.toml` before using it.

4.2 Standalone Python distribution

Distribution and import have different names
Install the Python distribution `linnet-py`, then import `linnet_py`. It requires Python 3.10 or newer. It is a standalone extension and is not a `symbolica.community` module. Its package version is independent of the Rust `linnet` version. Record both versions when diagnosing compatibility between Rust and Python code.

The Python binding focuses on DOT-backed graphs and mirrors typed identifiers and subgraphs:

```
from linnet_py import DotGraphBuilder

builder = DotGraphBuilder()
left = builder.add_node()
right = builder.add_node()
builder.add_edge(left, right)
graph = builder.build()

print(graph.dot())
```

`DotGraph` can also parse one graph from a string or file, or a set of graphs from a string. The exported classes include `Hedge`, `NodeIndex`, `EdgeIndex`, `Flow`, `Orientation`, graph attribute wrappers, `HedgePair`, `Subgraph`, `Cycle`, `OrientedCut`, `TraversalTree`, `DotGraph`, and `DotGraphBuilder`.

Mutable statement wrappers expose mapping-like access to DOT attributes. Consult the structured Python API for their lifetime and write-through behavior, and do not assume that a copied mapping remains attached to its graph.

5 Releases and change history

The Linnet changelog records changes to the Rust crate. The standalone `linnet-py` distribution has its own version and does not currently have a separate changelog in this repository.

The changelog records user-visible graph, parser, drawing, and algorithm changes. In particular, users moving across releases should check index ordering, DOT serialization, subgraph behavior, and feature-gated integration changes rather than assuming a serialized graph or incidental ordering is stable.

5.1 Choosing the matching documentation

The `latest` channel may describe unreleased behavior. Documentation under `snapshots/<tag>` is fixed to a tagged repository revision. Choose a snapshot whose component information lists your Rust crate version, and check the `linnet-py` version separately when using the Python bindings.

Before upgrading
Review changes to index ordering, DOT serialization, subgraph behavior, and enabled features. Re-validate serialized graphs and any code that depends on traversal or iteration order.

API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

`linnet`

Linnet's supported half-edge graph API. Algorithms are grouped in the order used by the manual and full Rustdoc.

`Half-edge graph`

```
Stores graph topology as paired half-edges with independent edge, vertex, and half-
edge payloads.
```

```
The main graph data structure, representing a graph using the half-edge
(or doubly connected edge list - DCEL) principle.

A `HedgeGraph` stores nodes and edges (represented as pairs of half-edges).
It is generic over the data associated with edges (`E`), nodes/vertices (`V`),
and the specific storage strategy used for nodes (`S`).

The half-edge representation allows for efficient traversal of graph topology,
such as iterating around faces (in planar embeddings), finding incident edges
to a node, or quickly accessing an edge's opposite half-edge.

# Type Parameters

- `E`: The type of data associated with each edge (or pair of half-edges).
- `V`: The type of data associated with each node (vertex).
- `S`: The node storage strategy, implementing the [`NodeStorage`] trait.
This determines how node data and their connectivity to half-edges
are stored. Defaults to [`DefaultNodeStore<V>`] (feature-selected; `nodestore-vec`
uses
[`nodestore::NodeStorageVec<V>`]).
```

```
#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord, Hash)]
#[cfg_attr(feature = "serde", derive(serde::Serialize, serde::Deserialize))]
#[cfg_attr(feature = "bincode",
derive(bincode_trait_derive::Encode, bincode_trait_derive::Decode))]
#[cfg_attr(feature = "rkyv",
derive(rkyv::Archive, rkyv::Serialize, rkyv::Deserialize))]
#[cfg_attr(feature = "rkyv", archive(check_bytes))]
#[doc =
" The main graph data structure, representing a graph using the half-edge"]
#[doc = " (or doubly connected edge list - DCEL) principle."] #[doc = ""]
#[doc =
" A `HedgeGraph` stores nodes and edges (represented as pairs of half-edges)."]
#[doc =
" It is generic over the data associated with edges (`E`), nodes/vertices (`V`),"]
#[doc = " and the specific storage strategy used for nodes (`S`)."]
#[doc = ""]
#[doc =
" The half-edge representation allows for efficient traversal of graph topology,"]
#[doc =
" such as iterating around faces (in planar embeddings), finding incident edges"]
#[doc = " to a node, or quickly accessing an edge\'s opposite half-edge."]
#[doc = ""] #[doc = " # Type Parameters"] #[doc = ""]
#[doc =
" - `E`: The type of data associated with each edge (or pair of half-edges)."]
```

```
#[doc = " - `V`: The type of data associated with each node (vertex)."]
#[doc =
" - `S`: The node storage strategy, implementing the [`NodeStorage`] trait."]
#[doc =
"   This determines how node data and their connectivity to half-edges"]
#[doc =
"   are stored. Defaults to [`DefaultNodeStore<V>`] (feature-selected; `nodestore-
vec` uses"]
#[doc = "   [`nodestore::NodeStorageVec<V>`])."] pub struct HedgeGraph < E, V,
H = NoData, S : NodeStorage < NodeData = V > = DefaultNodeStore < V > >
{
    pub(crate) hedge_data : HedgeVec < H > ,
    #[doc =
    " Internal storage for all half-edges, their data, and their topological"]
    #[doc =
    " relationships (e.g., opposite half-edge, next half-edge around a node)."]
    #[doc = " This is typically a [`SmartHedgeVec<E>`]."] pub(crate)
    edge_store : SmartEdgeVec < E > ,
    #[doc =
    " Storage for all nodes in the graph, including their data (`V`) and"]
    #[doc = " information about the half-edges incident to them."]
    #[doc =
    " The specific implementation is determined by the `S` type parameter."]
    pub node_store : S,
}
```

Reference feature matrix: `nodestore-vec, serde, bincode, drawing, symbolica` (item is available without an additional gate)

Supported usage

```
use linnet::half_edge::HedgeGraph;
```

`hedge_data`

Kind: `Field`

```
HedgeVec < H >
```

`edge_store`

Kind: `Field`

```
SmartEdgeVec < E >
```

```
Internal storage for all half-edges, their data, and their topological
relationships (e.g., opposite half-edge, next half-edge around a node).
This is typically a [`SmartHedgeVec<E>`].
```

`node_store`

Kind: `Field`

```
S
```

```
Storage for all nodes in the graph, including their data (`V`) and
information about the half-edges incident to them.
The specific implementation is determined by the `S` type parameter.
```

Exhaustive reference · Source

Graph builder

Builds a valid half-edge graph while assigning typed node, edge, and hedge identifiers.

A builder for programmatically constructing [`HedgeGraph`] instances.

This builder allows for the incremental addition of nodes and edges (both paired and external/dangling) before finalizing the graph structure.

Type Parameters

- `E`: The type of data to be associated with edges.
- `V`: The type of data to be associated with nodes.

Example

```rust,ignore
use linnet::half_edge::builder::HedgeGraphBuilder;
use linnet::half_edge::involution::{Flow, Orientation};

let mut builder = HedgeGraphBuilder::<&str, &str>::new();

let node0 = builder.add_node("Node_0_Data");
let node1 = builder.add_node("Node_1_Data");
let node2 = builder.add_node("Node_2_Data");

builder.add_edge(node0, node1, "Edge_01_Data", Orientation::Default);
builder.add_external_edge(node2, "External_Edge_Data", Orientation::Undirected,
Flow::Source);

// Assuming a NodeStorage type MyNodeStore is defined and implements NodeStorageOps
// let graph: HedgeGraph<&str, &str, MyNodeStore> = builder.build();
```

```
#[derive(Clone, Debug)]
#[cfg_attr(feature = "rkyv",
derive(rkyv::Archive, rkyv::Serialize, rkyv::Deserialize))]
#[doc =
" A builder for programmatically constructing [`HedgeGraph`] instances."]
#[doc = ""]
#[doc =
" This builder allows for the incremental addition of nodes and edges (both"]
#[doc =
" paired and external/dangling) before finalizing the graph structure."]
#[doc = ""] #[doc = " # Type Parameters"] #[doc = ""]
#[doc = " - `E`: The type of data to be associated with edges."]
#[doc = " - `V`: The type of data to be associated with nodes."] #[doc = ""]
#[doc = " # Example"] #[doc = ""] #[doc = " ```rust,ignore"]
#[doc = " use linnet::half_edge::builder::HedgeGraphBuilder;"]
#[doc = " use linnet::half_edge::involution::{Flow, Orientation};"]
#[doc = ""]
#[doc = " let mut builder = HedgeGraphBuilder::<&str, &str>::new();"]
#[doc = ""] #[doc = " let node0 = builder.add_node(\"Node_0_Data\");"]
#[doc = " let node1 = builder.add_node(\"Node_1_Data\");"]
#[doc = " let node2 = builder.add_node(\"Node_2_Data\");"] #[doc = ""]
#[doc =
" builder.add_edge(node0, node1, \"Edge_01_Data\", Orientation::Default);"]
```

```
#[doc =
" builder.add_external_edge(node2, \"External_Edge_Data\", Orientation::Undirected,
Flow::Source);"]
#[doc = ""]
#[doc =
" // Assuming a NodeStorage type MyNodeStore is defined and implements
NodeStorageOps"]
#[doc =
" // let graph: HedgeGraph<&str, &str, MyNodeStore> = builder.build();"]
#[doc = " ```"] pub struct HedgeGraphBuilder < E, V, H = NoData >
{
    hedge_data : HedgeVec < H > ,
    #[doc =
    " A list of nodes currently being built, stored as [`HedgeNodeBuilder`]
instances."]
    nodes : Vec < HedgeNodeBuilder < V > > ,
    #[doc =
    " The [`Involution`] structure managing the half-edges being added to the
graph."]
    pub(crate) involution : Involution < E > ,
}
```

Reference feature matrix: `nodestore-vec, serde, bincode, drawing, symbolica` (item is available without an additional gate)

Example 1

```
use linnet::half_edge::builder::HedgeGraphBuilder;
use linnet::half_edge::involution::{Flow, Orientation};

let mut builder = HedgeGraphBuilder::<&str, &str>::new();

let node0 = builder.add_node("Node_0_Data");
let node1 = builder.add_node("Node_1_Data");
let node2 = builder.add_node("Node_2_Data");

builder.add_edge(node0, node1, "Edge_01_Data", Orientation::Default);
builder.add_external_edge(node2, "External_Edge_Data", Orientation::Undirected,
Flow::Source);

// Assuming a NodeStorage type MyNodeStore is defined and implements NodeStorageOps
// let graph: HedgeGraph<&str, &str, MyNodeStore> = builder.build();
```

Supported usage

```
use linnet::half_edge::builder::HedgeGraphBuilder;
```

`hedge_data`

Kind: `Field`

```
HedgeVec < H >
```

`nodes`

Kind: `Field`

```
Vec < HedgeNodeBuilder < V > >
```

```
A list of nodes currently being built, stored as [`HedgeNodeBuilder`] instances.
```

`involution`

Kind: `Field`

`Involution < E >`

`The [`Involution`] structure managing the half-edges being added to the graph.`

Exhaustive reference · Source

`Bit-vector subgraph`

`Represents a subgraph as an efficient half-edge inclusion filter.`

`Represents a subgraph as an efficient half-edge inclusion filter.`

`pub type SuBitGraph = SubSet < Hedge > ;`

Reference feature matrix: `nodestore-vec, serde, bincode, drawing, symbolica` (item is available without an additional gate)

Supported usage

```
use linnet::half_edge::subgraph::SuBitGraph;
```

Exhaustive reference · Source

`Traversal tree`

`Records parent, root, and traversal information produced by breadth-first or depth-first graph walks.`

`Represents a traversal tree (e.g., DFS or BFS tree) derived from a [`HedgeGraph`].`

```
This structure uses a generic [`Forest`] to store the tree topology. Each "root" in
the
`Forest` corresponds to a node in the original `HedgeGraph`, and its data is a
[`TTRoot`]
enum indicating its role (Root, Child, or None) in this specific traversal. The nodes
within each tree of the `Forest` typically represent the half-edges that form the
traversal path.

# Type Parameters

- `P`: The type of [`ForestNodeStore`] used by the underlying `Forest`.
It stores `()` as `NodeData` because the tree nodes (representing half-edges)
don't need additional data beyond their structural role in the traversal tree.
Defaults to [`ParentPointerStore<()>`].
#[derive(Debug, Clone)]
#[cfg_attr(feature = "serde", derive(serde::Serialize, serde::Deserialize))]
#[doc =
" Represents a traversal tree (e.g., DFS or BFS tree) derived from a
[`HedgeGraph`]."]
#[doc = ""]
#[doc =
" This structure uses a generic [`Forest`] to store the tree topology. Each \"root\"
in the"]
#[doc =
" `Forest` corresponds to a node in the original `HedgeGraph`, and its data is a
[`TTRoot`]"]
#[doc =
" enum indicating its role (Root, Child, or None) in this specific traversal. The
nodes"]
```

```
#[doc =
" within each tree of the `Forest` typically represent the half-edges that form the"]
#[doc = " traversal path."] #[doc = ""] #[doc = " # Type Parameters"]
#[doc = ""]
#[doc =
" - `P`: The type of [`ForestNodeStore`] used by the underlying `Forest`."]
#[doc =
"   It stores `()` as `NodeData` because the tree nodes (representing half-edges)"]
#[doc =
"   don\'t need additional data beyond their structural role in the traversal tree."]
#[doc = "   Defaults to [`ParentPointerStore<()>`]."] pub struct
SimpleTraversalTree < P : ForestNodeStore < NodeData = () > =
ParentPointerStore < () > >
{
    #[doc =
    " The underlying forest structure representing the traversal tree(s)."]
    #[doc =
    " Each root in this forest is a graph node, and its data is its [`TTRoot`]
status."]
    #[doc = " Nodes within each tree of this forest represent half-edges."]
    forest : Forest < TTRoot, P > ,
    #[doc =
    " A bitmask representing the set of half-edges that constitute the actual"]
    #[doc = " edges of this traversal tree."] pub tree_subgraph : SuBitGraph,
}
```

Reference feature matrix: `nodestore-vec, serde, bincode, drawing, symbolica` (item is available without an additional gate)

Supported usage

```
use linnet::half_edge::tree::SimpleTraversalTree;
```

`forest`

Kind: `Field`

```
Forest < TTRoot, P >
```

```
The underlying forest structure representing the traversal tree(s).
Each root in this forest is a graph node, and its data is its [`TTRoot`] status.
Nodes within each tree of this forest represent half-edges.
```

`tree_subgraph`

Kind: `Field`

```
SuBitGraph
```

```
A bitmask representing the set of half-edges that constitute the actual
edges of this traversal tree.
```

Exhaustive reference · Source

`DOT-backed graph`

```
Combines a half-edge graph with DOT attributes and parser/serializer support.
```

```
Combines a half-edge graph with DOT attributes and parser/serializer support.
```

```
#[derive(Debug, Clone, PartialEq, Eq)]
#[cfg_attr(feature = "rkyv",
derive(rkyv::Archive, rkyv::Serialize, rkyv::Deserialize))]
#[cfg_attr(feature = "rkyv", archive(check_bytes))] pub struct DotGraph < N :
```

```
NodeStorage < NodeData = DotVertexData > = DefaultNodeStore < DotVertexData >
>
{
    pub global_data : GlobalData, pub graph : HedgeGraph < DotEdgeData,
    DotVertexData, DotHedgeData, N > ,
}
```

Reference feature matrix: `nodestore-vec, serde, bincode, drawing, symbolica` (item is available without an additional gate)

Supported usage

```
use linnet::parser::DotGraph;
```

`global_data`

Kind: `Field`

```
GlobalData
```

`graph`

Kind: `Field`

```
HedgeGraph < DotEdgeData, DotVertexData, DotHedgeData, N >
```

Exhaustive reference · Source

`linnet-py`

```
Exports registered by linnet-py.
```

`Cycle`

```
Cycle represented as a subgraph and optional loop count.
```

```
Cycle represented as a subgraph and optional loop count.
```

```
class Cycle:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import Cycle
help(Cycle)
```

`filter`

Kind: `Getter`

```
def filter(self) -> Subgraph:
```

```
Subgraph filter for this cycle.
```

`loop_count`

Kind: `Getter`

```
def loop_count(self) -> typing.Optional[builtins.int]:
```

```
Optional loop count for this cycle.
```

`__repr__`

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

DotEdgeData

DOT edge data (attributes + optional edge id).

DOT edge data (attributes + optional edge id).

```
class DotEdgeData:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import DotEdgeData
help(DotEdgeData)
```

statements

Kind: `Getter`

```
def statements(self) -> EdgeStatements:
```

Edge attributes as a dict-like proxy.

local_statements

Kind: `Getter`

```
def local_statements(self) -> EdgeStatements:
```

Local edge attributes as a dict-like proxy.

edge_id

Kind: `Getter`

```
def edge_id(self) -> typing.Optional[EdgeIndex]:
```

Optional edge id.

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, statements: typing.Optional[typing.Mapping[builtins.str,
builtins.str]] = None, local_statements: typing.Optional[typing.Mapping[builtins.str,
builtins.str]] = None, edge_id: typing.Optional[builtins.int] = None) -> DotEdgeData:
```

Create edge data from fields.

statements

Kind: `Parameter`

```
statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: `None`

local_statements

Kind: `Parameter`

```
local_statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: `None`

edge_id

Kind: `Parameter`

```
edge_id: typing.Optional[builtins.int] = None
```

Default: `None`

`add_statement`
Kind: `Method`

```
def add_statement(self, key: builtins.str, value: builtins.str) -> None:
```

```
Insert or update an attribute statement.
```

`key`
Kind: `Parameter`

```
key: builtins.str
```

`value`
Kind: `Parameter`

```
value: builtins.str
```

`__repr__`
Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

`DotGraph`
```
DOT-backed hedge graph.
```

```
DOT-backed hedge graph.
```

```
class DotGraph:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import DotGraph
help(DotGraph)
```

`global_data`
Kind: `Getter`

```
def global_data(self) -> GlobalData:
```

```
Global graph attributes.
```

`global_data`
Kind: `Setter`

```
def global_data(self, value: GlobalData) -> None:
```

`value`
Kind: `Parameter`

```
value: GlobalData
```

`from_string`
Kind: `AssociatedFunction`

```
def from_string(cls, s: builtins.str) -> DotGraph:
```

Parse a DOT string into a graph.

s
Kind: Parameter

s: builtins.str

from_string_set
Kind: AssociatedFunction

def from_string_set(cls, s: builtins.str) -> builtins.list[DotGraph]:

Parse a DOT string into multiple graphs.

s
Kind: Parameter

s: builtins.str

from_file
Kind: AssociatedFunction

def from_file(cls, path: builtins.str) -> DotGraph:

Parse a DOT file into a graph.

path
Kind: Parameter

path: builtins.str

debug_dot
Kind: Method

def debug_dot(self) -> builtins.str:

Serialize graph to DOT for debugging.

dot
Kind: Method

def dot(self) -> builtins.str:

Serialize the full graph to DOT.

dot_of
Kind: Method

def dot_of(self, subgraph: Subgraph) -> builtins.str:

Serialize a subgraph to DOT.

subgraph
Kind: Parameter

subgraph: Subgraph

n_nodes
Kind: Method

def n_nodes(self) -> builtins.int:

Number of nodes.

n_edges

Kind: Method

```
def n_edges(self) -> builtins.int:
```

Number of edges.

n_hedges

Kind: Method

```
def n_hedges(self) -> builtins.int:
```

Number of hedges.

n_externals

Kind: Method

```
def n_externals(self) -> builtins.int:
```

Number of external hedges.

n_internals

Kind: Method

```
def n_internals(self) -> builtins.int:
```

Number of internal hedges.

full_filter

Kind: Method

```
def full_filter(self) -> Subgraph:
```

Subgraph including all hedges.

empty_subgraph

Kind: Method

```
def empty_subgraph(self) -> Subgraph:
```

Empty subgraph of this graph.

iter_edges

Kind: Method

```
def iter_edges(self) -> builtins.list[tuple[HedgePair, EdgeIndex, EdgeData]]:
```

Iterate edges in the full graph.

iter_edges_of

Kind: Method

```
def iter_edges_of(self, subgraph: Subgraph) -> builtins.list[tuple[HedgePair,
EdgeIndex, EdgeData]]:
```

Iterate edges within a subgraph.

subgraph

Kind: Parameter

```
subgraph: Subgraph
```

iter_nodes

Kind: Method

def iter_nodes(self) -> builtins.list[tuple[NodeIndex, builtins.list[Hedge], DotVertexData]]:

Iterate nodes in the full graph.

iter_nodes_of

Kind: Method

def iter_nodes_of(self, subgraph: Subgraph) -> builtins.list[tuple[NodeIndex, builtins.list[Hedge], DotVertexData]]:

Iterate nodes within a subgraph.

subgraph

Kind: Parameter

subgraph: Subgraph

connected_components

Kind: Method

def connected_components(self, subgraph: Subgraph) -> builtins.list[Subgraph]:

Connected components of a subgraph.

subgraph

Kind: Parameter

subgraph: Subgraph

count_connected_components

Kind: Method

def count_connected_components(self, subgraph: Subgraph) -> builtins.int:

Count connected components.

subgraph

Kind: Parameter

subgraph: Subgraph

is_connected

Kind: Method

def is_connected(self, subgraph: Subgraph) -> builtins.bool:

Whether a subgraph is connected.

subgraph

Kind: Parameter

subgraph: Subgraph

depth_first_traverse

Kind: Method

def depth_first_traverse(self, subgraph: Subgraph, root_node: NodeIndex, include_hedge: typing.Optional[Hedge]) -> TraversalTree:

Depth-first traversal from a root node.

subgraph

Kind: Parameter

subgraph: Subgraph

root_node

Kind: Parameter

root_node: NodeIndex

include_hedge

Kind: Parameter

include_hedge: typing.Optional[Hedge]

breadth_first_traverse

Kind: Method

def breadth_first_traverse(self, subgraph: Subgraph, root_node: NodeIndex, include_hedge: typing.Optional[Hedge]) -> TraversalTree:

Breadth-first traversal from a root node.

subgraph

Kind: Parameter

subgraph: Subgraph

root_node

Kind: Parameter

root_node: NodeIndex

include_hedge

Kind: Parameter

include_hedge: typing.Optional[Hedge]

bridges

Kind: Method

def bridges(self) -> Subgraph:

Bridges in the full graph.

bridges_of

Kind: Method

def bridges_of(self, subgraph: Subgraph) -> Subgraph:

Bridges within a subgraph.

subgraph

Kind: Parameter

subgraph: Subgraph

cycle_basis

Kind: Method

def cycle_basis(self) -> tuple[builtins.list[Cycle], Subgraph]:

Cycle basis of the full graph.

cycle_basis_of

Kind: Method

def cycle_basis_of(self, subgraph: Subgraph) -> tuple[builtins.list[Cycle], Subgraph]:

Cycle basis of a subgraph.

subgraph

Kind: Parameter

subgraph: Subgraph

all_spanning_forests

Kind: Method

def all_spanning_forests(self) -> builtins.list[Subgraph]:

All spanning forests of the full graph.

all_spanning_forests_of

Kind: Method

def all_spanning_forests_of(self, subgraph: Subgraph) -> builtins.list[Subgraph]:

All spanning forests of a subgraph.

subgraph

Kind: Parameter

subgraph: Subgraph

combine_to_single_hedgenode

Kind: Method

def combine_to_single_hedgenode(self, nodes: typing.Sequence[NodeIndex]) -> HedgeNode:

Combine nodes into a single hedge node.

nodes

Kind: Parameter

nodes: typing.Sequence[NodeIndex]

all_cuts

Kind: Method

def all_cuts(self, source: HedgeNode, target: HedgeNode) -> builtins.list[tuple[Subgraph, OrientedCut, Subgraph]]:

All cuts between two hedge nodes.

source

Kind: Parameter

source: HedgeNode

target

Kind: Parameter

target: HedgeNode

all_cuts_from_ids

Kind: Method

```
def all_cuts_from_ids(self, source: typing.Sequence[NodeIndex], target:
typing.Sequence[NodeIndex]) -> builtins.list[tuple[Subgraph, OrientedCut, Subgraph]]:
```

All cuts between two sets of node indices.

source

Kind: Parameter

```
source: typing.Sequence[NodeIndex]
```

target

Kind: Parameter

```
target: typing.Sequence[NodeIndex]
```

contract_subgraph

Kind: Method

```
def contract_subgraph(self, subgraph: Subgraph, node_data_merge:
typing.Optional[DotVertexData] = None) -> None:
```

Contract a subgraph into a single node, deleting its edges.

subgraph

Kind: Parameter

```
subgraph: Subgraph
```

node_data_merge

Kind: Parameter

```
node_data_merge: typing.Optional[DotVertexData] = None
```

Default: None

join

Kind: Method

```
def join(self, other: DotGraph, matching_fn: typing.Any, merge_fn: typing.Any) ->
DotGraph:
```

Join two graphs, matching dangling edges via a Python callback.

other

Kind: Parameter

```
other: DotGraph
```

matching_fn

Kind: Parameter

```
matching_fn: typing.Any
```

merge_fn

Kind: Parameter

```
merge_fn: typing.Any
```

join_mut

Kind: Method

```
def join_mut(self, other: DotGraph, matching_fn: typing.Any, merge_fn: typing.Any) ->
None:
```

In-place join, matching dangling edges via a Python callback.

other

Kind: Parameter

```
other: DotGraph
```

matching_fn

Kind: Parameter

```
matching_fn: typing.Any
```

merge_fn

Kind: Parameter

```
merge_fn: typing.Any
```

extract

Kind: Method

```
def extract(self, subgraph: Subgraph, split_edge_fn: typing.Any, internal_data:
typing.Any, split_node: typing.Any, owned_node: typing.Any) -> DotGraph:
```

Extract a subgraph with Python callbacks to transform edge/node data.

subgraph

Kind: Parameter

```
subgraph: Subgraph
```

split_edge_fn

Kind: Parameter

```
split_edge_fn: typing.Any
```

internal_data

Kind: Parameter

```
internal_data: typing.Any
```

split_node

Kind: Parameter

```
split_node: typing.Any
```

owned_node

Kind: Parameter

```
owned_node: typing.Any
```

__getitem__

Kind: Method

```
def __getitem__(self, key: Hedge) -> DotHedgeData:
```

key

Kind: `Parameter`

```
key: Hedge
```

__getitem__

Kind: `Method`

```
def __getitem__(self, key: NodeIndex) -> DotVertexData:
```

key

Kind: `Parameter`

```
key: NodeIndex
```

__getitem__

Kind: `Method`

```
def __getitem__(self, key: EdgeIndex) -> DotEdgeData:
```

key

Kind: `Parameter`

```
key: EdgeIndex
```

Exhaustive reference · Source

DotGraphBuilder

```
Builder for constructing DOT graphs programmatically.
```

```
Builder for constructing DOT graphs programmatically.
```

```
class DotGraphBuilder:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import DotGraphBuilder
help(DotGraphBuilder)
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls) -> DotGraphBuilder:
```

```
Create a new, empty graph builder.
```

add_node

Kind: `Method`

```
def add_node(self, data: typing.Optional[DotVertexData] = None) -> NodeIndex:
```

```
Add a node and return its index.
```

data

Kind: `Parameter`

```
data: typing.Optional[DotVertexData] = None
```

Default: `None`

add_edge

Kind: Method

```
def add_edge(self, source: NodeIndex, sink: NodeIndex, data:
typing.Optional[DotEdgeData] = None, orientation: typing.Optional[Orientation] =
None, source_hedge: typing.Optional[DotHedgeData] = None, sink_hedge:
typing.Optional[DotHedgeData] = None) -> None:
```

Add an edge between two nodes.

source

Kind: Parameter

```
source: NodeIndex
```

sink

Kind: Parameter

```
sink: NodeIndex
```

data

Kind: Parameter

```
data: typing.Optional[DotEdgeData] = None
```

Default: None

orientation

Kind: Parameter

```
orientation: typing.Optional[Orientation] = None
```

Default: None

source_hedge

Kind: Parameter

```
source_hedge: typing.Optional[DotHedgeData] = None
```

Default: None

sink_hedge

Kind: Parameter

```
sink_hedge: typing.Optional[DotHedgeData] = None
```

Default: None

add_external_edge

Kind: Method

```
def add_external_edge(self, source: NodeIndex, data: typing.Optional[DotEdgeData] =
None, orientation: typing.Optional[Orientation] = None, flow: typing.Optional[Flow] =
None, hedge: typing.Optional[DotHedgeData] = None) -> None:
```

Add a dangling (external) edge incident to a node.

source

Kind: Parameter

```
source: NodeIndex
```

data
Kind: Parameter

```
data: typing.Optional[DotEdgeData] = None
```

Default: None

orientation
Kind: Parameter

```
orientation: typing.Optional[Orientation] = None
```

Default: None

flow
Kind: Parameter

```
flow: typing.Optional[Flow] = None
```

Default: None

hedge
Kind: Parameter

```
hedge: typing.Optional[DotHedgeData] = None
```

Default: None

build
Kind: Method

```
def build(self) -> DotGraph:
```

```
Build the graph and reset the builder.
```

Exhaustive reference · Source

DotHedgeData
DOT hedge (half-edge) data.

DOT hedge (half-edge) data.

```
class DotHedgeData:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import DotHedgeData
help(DotHedgeData)
```

statement
Kind: Getter

```
def statement(self) -> typing.Optional[builtins.str]:
```

```
Optional statement string.
```

id
Kind: Getter

```
def id(self) -> typing.Optional[Hedge]:
```

```
Optional hedge id.
```

port_label

Kind: Getter

```
def port_label(self) -> typing.Optional[builtins.str]:
```

Optional port label.

compasspt

Kind: Getter

```
def compasspt(self) -> typing.Optional[builtins.str]:
```

Optional compass point as a string.

__new__

Kind: AssociatedFunction

```
def __new__(cls, statement: typing.Optional[builtins.str] = None, id:
typing.Optional[builtins.int] = None, port_label: typing.Optional[builtins.str] =
None, compasspt: typing.Optional[builtins.str] = None) -> DotHedgeData:
```

Create hedge data from fields.

statement

Kind: Parameter

```
statement: typing.Optional[builtins.str] = None
```

Default: None

id

Kind: Parameter

```
id: typing.Optional[builtins.int] = None
```

Default: None

port_label

Kind: Parameter

```
port_label: typing.Optional[builtins.str] = None
```

Default: None

compasspt

Kind: Parameter

```
compasspt: typing.Optional[builtins.str] = None
```

Default: None

__repr__

Kind: Method

```
def __repr__(self) -> builtins.str:
```

Debug-style representation.

Exhaustive reference · Source

DotVertexData

DOT vertex data (name, optional index, and attribute statements).

DOT vertex data (name, optional index, and attribute statements).

class DotVertexData:

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import DotVertexData
help(DotVertexData)
```

name

Kind: `Getter`

```
def name(self) -> typing.Optional[builtins.str]:
```

Optional vertex name.

index

Kind: `Getter`

```
def index(self) -> typing.Optional[NodeIndex]:
```

Optional node index.

statements

Kind: `Getter`

```
def statements(self) -> NodeStatements:
```

Attribute statements as a dict-like proxy.

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, name: typing.Optional[builtins.str] = None, index:
typing.Optional[builtins.int] = None, statements:
typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None) -> DotVertexData:
```

Create vertex data from fields.

name

Kind: `Parameter`

```
name: typing.Optional[builtins.str] = None
```

Default: `None`

index

Kind: `Parameter`

```
index: typing.Optional[builtins.int] = None
```

Default: `None`

statements

Kind: `Parameter`

```
statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: `None`

add_statement

Kind: `Method`

```
def add_statement(self, key: builtins.str, value: builtins.str) -> None:
```

Insert or update an attribute statement.

key
Kind: Parameter

key: builtins.str

value
Kind: Parameter

value: builtins.str

__repr__
Kind: Method

def __repr__(self) -> builtins.str:

Debug-style representation.

Exhaustive reference · Source

EdgeData
Edge data with orientation.

Edge data with orientation.

class EdgeData:

Requires features: extension-module, abi3-py310

Inspect the registered export

```
from linnet_py import EdgeData
help(EdgeData)
```

orientation
Kind: Getter

def orientation(self) -> Orientation:

Orientation of this edge.

data
Kind: Getter

def data(self) -> DotEdgeData:

DOT edge data.

__new__
Kind: AssociatedFunction

def __new__(cls, orientation: typing.Any, data: typing.Any) -> EdgeData:

Create edge data from orientation and DotEdgeData.

orientation
Kind: Parameter

orientation: typing.Any

data
Kind: Parameter

```
data: typing.Any
```

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

EdgeIndex

```
Edge index identifier.
```

```
Edge index identifier.
```

```
class EdgeIndex:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import EdgeIndex
help(EdgeIndex)
```

value

Kind: `Getter`

```
def value(self) -> builtins.int:
```

```
Numeric value of this edge index.
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, value: builtins.int) -> EdgeIndex:
```

```
Create an edge index from a zero-based index.
```

value

Kind: `Parameter`

```
value: builtins.int
```

__int__

Kind: `Method`

```
def __int__(self) -> builtins.int:
```

```
Convert to int.
```

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

EdgeStatements

```
Dict-like proxy for edge attribute statements.
```

```
Dict-like proxy for edge attribute statements.
```

```
class EdgeStatements:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import EdgeStatements
help(EdgeStatements)
```

__getitem__
Kind: `Method`

```
def __getitem__(self, key: builtins.str) -> builtins.str:
```

key
Kind: `Parameter`

```
key: builtins.str
```

__setitem__
Kind: `Method`

```
def __setitem__(self, key: builtins.str, value: builtins.str) -> None:
```

key
Kind: `Parameter`

```
key: builtins.str
```

value
Kind: `Parameter`

```
value: builtins.str
```

__delitem__
Kind: `Method`

```
def __delitem__(self, key: builtins.str) -> None:
```

key
Kind: `Parameter`

```
key: builtins.str
```

__contains__
Kind: `Method`

```
def __contains__(self, key: builtins.str) -> builtins.bool:
```

key
Kind: `Parameter`

```
key: builtins.str
```

__len__
Kind: `Method`

```
def __len__(self) -> builtins.int:
```

__iter__
Kind: `Method`

```
def __iter__(self) -> typing.Iterator[builtins.str]:
```

get

Kind: Method

```
def get(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: Parameter

```
default: typing.Any = None
```

Default: None

keys

Kind: Method

```
def keys(self) -> builtins.list[builtins.str]:
```

values

Kind: Method

```
def values(self) -> builtins.list[builtins.str]:
```

items

Kind: Method

```
def items(self) -> builtins.list[tuple[builtins.str, builtins.str]]:
```

clear

Kind: Method

```
def clear(self) -> None:
```

update

Kind: Method

```
def update(self, other: typing.Mapping[builtins.str, builtins.str]) -> None:
```

other

Kind: Parameter

```
other: typing.Mapping[builtins.str, builtins.str]
```

pop

Kind: Method

```
def pop(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: Parameter

```
default: typing.Any = None
```

Default: `None`

popitem

Kind: `Method`

```
def popitem(self) -> tuple[builtins.str, builtins.str]:
```

setdefault

Kind: `Method`

```
def setdefault(self, key: builtins.str, default: typing.Optional[builtins.str] =
None) -> builtins.str:
```

key

Kind: `Parameter`

```
key: builtins.str
```

default

Kind: `Parameter`

```
default: typing.Optional[builtins.str] = None
```

Default: `None`

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

Exhaustive reference · Source

Flow

```
Flow direction (Source/Sink).
```

```
Flow direction (Source/Sink).
```

```
class Flow:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import Flow
help(Flow)
```

source

Kind: `AssociatedFunction`

```
def source() -> Flow:
```

```
Flow::Source.
```

sink

Kind: `AssociatedFunction`

```
def sink() -> Flow:
```

```
Flow::Sink.
```

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

GlobalData

```
Global graph attributes (graph/node/edge statements).
```

```
Global graph attributes (graph/node/edge statements).
```

```
class GlobalData:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import GlobalData
help(GlobalData)
```

name

Kind: `Getter`

```
def name(self) -> builtins.str:
```

```
Graph name.
```

name

Kind: `Setter`

```
def name(self, value: builtins.str) -> None:
```

value

Kind: `Parameter`

```
value: builtins.str
```

statements

Kind: `Getter`

```
def statements(self) -> GlobalStatements:
```

```
Graph-level statements.
```

statements

Kind: `Setter`

```
def statements(self, value: builtins.dict[builtins.str, builtins.str]) -> None:
```

value

Kind: `Parameter`

```
value: builtins.dict[builtins.str, builtins.str]
```

edge_statements

Kind: `Getter`

```
def edge_statements(self) -> GlobalStatements:
```

```
Default edge statements.
```

edge_statements

Kind: `Setter`

```
def edge_statements(self, value: builtins.dict[builtins.str, builtins.str]) -> None:
```

value

Kind: Parameter

```
value: builtins.dict[builtins.str, builtins.str]
```

node_statements

Kind: Getter

```
def node_statements(self) -> GlobalStatements:
```

Default node statements.

node_statements

Kind: Setter

```
def node_statements(self, value: builtins.dict[builtins.str, builtins.str]) -> None:
```

value

Kind: Parameter

```
value: builtins.dict[builtins.str, builtins.str]
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, name: typing.Optional[builtins.str] = None, statements:
typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None, edge_statements:
typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None, node_statements:
typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None) -> GlobalData:
```

Create global data from fields.

name

Kind: Parameter

```
name: typing.Optional[builtins.str] = None
```

Default: None

statements

Kind: Parameter

```
statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: None

edge_statements

Kind: Parameter

```
edge_statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: None

node_statements

Kind: Parameter

```
node_statements: typing.Optional[typing.Mapping[builtins.str, builtins.str]] = None
```

Default: None

add_statement

Kind: Method

```
def add_statement(self, key: builtins.str, value: builtins.str) -> None:
```

Insert or update a graph-level statement.

key

Kind: Parameter

```
key: builtins.str
```

value

Kind: Parameter

```
value: builtins.str
```

add_edge_statement

Kind: Method

```
def add_edge_statement(self, key: builtins.str, value: builtins.str) -> None:
```

Insert or update a default edge statement.

key

Kind: Parameter

```
key: builtins.str
```

value

Kind: Parameter

```
value: builtins.str
```

add_node_statement

Kind: Method

```
def add_node_statement(self, key: builtins.str, value: builtins.str) -> None:
```

Insert or update a default node statement.

key

Kind: Parameter

```
key: builtins.str
```

value

Kind: Parameter

```
value: builtins.str
```

__repr__

Kind: Method

```
def __repr__(self) -> builtins.str:
```

Debug-style representation.

Exhaustive reference · Source

GlobalStatements

Dict-like proxy for global statements.

Dict-like proxy for global statements.

class GlobalStatements:

Requires features: extension-module, abi3-py310

Inspect the registered export

```
from linnet_py import GlobalStatements
help(GlobalStatements)
```

__getitem__
Kind: Method

def __getitem__(self, key: builtins.str) -> builtins.str:

key
Kind: Parameter

key: builtins.str

__setitem__
Kind: Method

def __setitem__(self, key: builtins.str, value: builtins.str) -> None:

key
Kind: Parameter

key: builtins.str

value
Kind: Parameter

value: builtins.str

__delitem__
Kind: Method

def __delitem__(self, key: builtins.str) -> None:

key
Kind: Parameter

key: builtins.str

__contains__
Kind: Method

def __contains__(self, key: builtins.str) -> builtins.bool:

key
Kind: Parameter

key: builtins.str

__len__
Kind: Method

def __len__(self) -> builtins.int:

__iter__

Kind: Method

```
def __iter__(self) -> typing.Iterator[builtins.str]:
```

get

Kind: Method

```
def get(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: Parameter

```
default: typing.Any = None
```

Default: None

keys

Kind: Method

```
def keys(self) -> builtins.list[builtins.str]:
```

values

Kind: Method

```
def values(self) -> builtins.list[builtins.str]:
```

items

Kind: Method

```
def items(self) -> builtins.list[tuple[builtins.str, builtins.str]]:
```

clear

Kind: Method

```
def clear(self) -> None:
```

update

Kind: Method

```
def update(self, other: typing.Mapping[builtins.str, builtins.str]) -> None:
```

other

Kind: Parameter

```
other: typing.Mapping[builtins.str, builtins.str]
```

pop

Kind: Method

```
def pop(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: `Parameter`

```
default: typing.Any = None
```

Default: `None`

popitem

Kind: `Method`

```
def popitem(self) -> tuple[builtins.str, builtins.str]:
```

setdefault

Kind: `Method`

```
def setdefault(self, key: builtins.str, default: typing.Optional[builtins.str] =
None) -> builtins.str:
```

key

Kind: `Parameter`

```
key: builtins.str
```

default

Kind: `Parameter`

```
default: typing.Optional[builtins.str] = None
```

Default: `None`

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

Exhaustive reference · Source

Hedge

```
Half-edge identifier.
```

```
Half-edge identifier.
```

```
class Hedge:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import Hedge
help(Hedge)
```

value

Kind: `Getter`

```
def value(self) -> builtins.int:
```

```
Numeric value of this hedge.
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, value: builtins.int) -> Hedge:
```

Create a hedge from a zero-based index.

value

Kind: Parameter

value: builtins.int

__int__

Kind: Method

def __int__(self) -> builtins.int:

Convert to int.

__repr__

Kind: Method

def __repr__(self) -> builtins.str:

Debug-style representation.

Exhaustive reference · Source

HedgeNode

Hedge node with internal graph and hairs.

Hedge node with internal graph and hairs.

class HedgeNode:

Requires features: extension-module, abi3-py310

Inspect the registered export

```
from linnet_py import HedgeNode
help(HedgeNode)
```

internal_graph

Kind: Getter

def internal_graph(self) -> Subgraph:

Internal subgraph.

hairs

Kind: Getter

def hairs(self) -> Subgraph:

Hair subgraph.

__new__

Kind: AssociatedFunction

def __new__(cls, internal_graph: Subgraph, hairs: Subgraph) -> HedgeNode:

Create a hedge node from internal graph and hairs subgraphs.

internal_graph

Kind: Parameter

internal_graph: Subgraph

hairs

Kind: `Parameter`

```
hairs: Subgraph
```

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

HedgePair

```
Hedge pairing (paired, unpaired, split).
```

```
Hedge pairing (paired, unpaired, split).
```

```
class HedgePair:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import HedgePair
help(HedgePair)
```

kind

Kind: `Getter`

```
def kind(self) -> builtins.str:
```

```
Kind of hedge pair: \"paired\", \"unpaired\", or \"split\".
```

source

Kind: `Getter`

```
def source(self) -> typing.Optional[Hedge]:
```

```
Source hedge (paired/split).
```

sink

Kind: `Getter`

```
def sink(self) -> typing.Optional[Hedge]:
```

```
Sink hedge (paired/split).
```

hedge

Kind: `Getter`

```
def hedge(self) -> typing.Optional[Hedge]:
```

```
Hedge (unpaired only).
```

flow

Kind: `Getter`

```
def flow(self) -> typing.Optional[Flow]:
```

```
Flow for unpaired hedge.
```

split

Kind: `Getter`

```
def split(self) -> typing.Optional[Flow]:
```

```
Which side is split for split edges.
```

__repr__
Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

NodeIndex
```
Node index identifier.
```

```
Node index identifier.
```

```
class NodeIndex:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import NodeIndex
help(NodeIndex)
```

value
Kind: `Getter`

```
def value(self) -> builtins.int:
```

```
Numeric value of this node index.
```

__new__
Kind: `AssociatedFunction`

```
def __new__(cls, value: builtins.int) -> NodeIndex:
```

```
Create a node index from a zero-based index.
```

value
Kind: `Parameter`

```
value: builtins.int
```

__int__
Kind: `Method`

```
def __int__(self) -> builtins.int:
```

```
Convert to int.
```

__repr__
Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

NodeStatements
```
Dict-like proxy for node attribute statements.
```

```
Dict-like proxy for node attribute statements.
```

```
class NodeStatements:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import NodeStatements
help(NodeStatements)
```

`__getitem__`

Kind: `Method`

```
def __getitem__(self, key: builtins.str) -> builtins.str:
```

`key`

Kind: `Parameter`

```
key: builtins.str
```

`__setitem__`

Kind: `Method`

```
def __setitem__(self, key: builtins.str, value: builtins.str) -> None:
```

`key`

Kind: `Parameter`

```
key: builtins.str
```

`value`

Kind: `Parameter`

```
value: builtins.str
```

`__delitem__`

Kind: `Method`

```
def __delitem__(self, key: builtins.str) -> None:
```

`key`

Kind: `Parameter`

```
key: builtins.str
```

`__contains__`

Kind: `Method`

```
def __contains__(self, key: builtins.str) -> builtins.bool:
```

`key`

Kind: `Parameter`

```
key: builtins.str
```

`__len__`

Kind: `Method`

```
def __len__(self) -> builtins.int:
```

`__iter__`

Kind: `Method`

```
def __iter__(self) -> typing.Iterator[builtins.str]:
```

get

Kind: Method

```
def get(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: Parameter

```
default: typing.Any = None
```

Default: None

keys

Kind: Method

```
def keys(self) -> builtins.list[builtins.str]:
```

values

Kind: Method

```
def values(self) -> builtins.list[builtins.str]:
```

items

Kind: Method

```
def items(self) -> builtins.list[tuple[builtins.str, builtins.str]]:
```

clear

Kind: Method

```
def clear(self) -> None:
```

update

Kind: Method

```
def update(self, other: typing.Mapping[builtins.str, builtins.str]) -> None:
```

other

Kind: Parameter

```
other: typing.Mapping[builtins.str, builtins.str]
```

pop

Kind: Method

```
def pop(self, key: builtins.str, default: typing.Any = None) -> typing.Any:
```

key

Kind: Parameter

```
key: builtins.str
```

default

Kind: `Parameter`

```
default: typing.Any = None
```

Default: `None`

popitem

Kind: `Method`

```
def popitem(self) -> tuple[builtins.str, builtins.str]:
```

setdefault

Kind: `Method`

```
def setdefault(self, key: builtins.str, default: typing.Optional[builtins.str] =
None) -> builtins.str:
```

key

Kind: `Parameter`

```
key: builtins.str
```

default

Kind: `Parameter`

```
default: typing.Optional[builtins.str] = None
```

Default: `None`

__repr__

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

Exhaustive reference · Source

Orientation

```
Edge orientation (Default/Reversed/Undirected).
```

```
Edge orientation (Default/Reversed/Undirected).
```

```
class Orientation:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import Orientation
help(Orientation)
```

default

Kind: `AssociatedFunction`

```
def default() -> Orientation:
```

```
Orientation::Default.
```

reversed

Kind: `AssociatedFunction`

```
def reversed() -> Orientation:
```

Orientation::Reversed.

undirected

Kind: AssociatedFunction

```
def undirected() -> Orientation:
```

Orientation::Undirected.

__repr__

Kind: Method

```
def __repr__(self) -> builtins.str:
```

Debug-style representation.

Exhaustive reference · Source

OrientedCut

Oriented cut represented by left/right subgraphs.

Oriented cut represented by left/right subgraphs.

```
class OrientedCut:
```

Requires features: extension-module, abi3-py310

Inspect the registered export

```
from linnet_py import OrientedCut
help(OrientedCut)
```

left

Kind: Getter

```
def left(self) -> Subgraph:
```

Left side of the cut.

right

Kind: Getter

```
def right(self) -> Subgraph:
```

Right side of the cut.

__repr__

Kind: Method

```
def __repr__(self) -> builtins.str:
```

Debug-style representation.

Exhaustive reference · Source

Subgraph

Subgraph represented as a bitset of hedges.

Subgraph represented as a bitset of hedges.

```
class Subgraph:
```

Requires features: extension-module, abi3-py310

Inspect the registered export

```
from linnet_py import Subgraph
help(Subgraph)
```

empty

Kind: AssociatedFunction

```
def empty(cls, size: builtins.int) -> Subgraph:
```

```
Create an empty subgraph of the given size (number of hedges).
```

size

Kind: Parameter

```
size: builtins.int
```

full

Kind: AssociatedFunction

```
def full(cls, size: builtins.int) -> Subgraph:
```

```
Create a full subgraph including all hedges.
```

size

Kind: Parameter

```
size: builtins.int
```

from_hedges

Kind: AssociatedFunction

```
def from_hedges(cls, size: builtins.int, hedges: typing.Sequence[Hedge]) -> Subgraph:
```

```
Create a subgraph from a list of hedges.
```

size

Kind: Parameter

```
size: builtins.int
```

hedges

Kind: Parameter

```
hedges: typing.Sequence[Hedge]
```

to_hedges

Kind: Method

```
def to_hedges(self) -> builtins.list[Hedge]:
```

```
List included hedges.
```

size

Kind: Method

```
def size(self) -> builtins.int:
```

```
Total number of hedges in the parent graph.
```

n_included

Kind: Method

```
def n_included(self) -> builtins.int:
```

Number of included hedges.

includes

Kind: Method

def includes(self, hedge: typing.Any) -> builtins.bool:

Whether a hedge is included.

hedge

Kind: Parameter

hedge: typing.Any

union

Kind: Method

def union(self, other: Subgraph) -> Subgraph:

Union with another subgraph.

other

Kind: Parameter

other: Subgraph

intersection

Kind: Method

def intersection(self, other: Subgraph) -> Subgraph:

Intersection with another subgraph.

other

Kind: Parameter

other: Subgraph

sym_diff

Kind: Method

def sym_diff(self, other: Subgraph) -> Subgraph:

Symmetric difference with another subgraph.

other

Kind: Parameter

other: Subgraph

subtract

Kind: Method

def subtract(self, other: Subgraph) -> Subgraph:

Subtract another subgraph.

other

Kind: Parameter

other: Subgraph

`__repr__`

Kind: `Method`

```
def __repr__(self) -> builtins.str:
```

```
Debug-style representation.
```

Exhaustive reference · Source

`TraversalTree`

```
Traversal tree produced by DFS/BFS.
```

```
Traversal tree produced by DFS/BFS.
```

```
class TraversalTree:
```

Requires features: `extension-module, abi3-py310`

Inspect the registered export

```
from linnet_py import TraversalTree
help(TraversalTree)
```

`tree_subgraph`

Kind: `Method`

```
def tree_subgraph(self) -> Subgraph:
```

```
Subgraph corresponding to the tree edges.
```

`node_order`

Kind: `Method`

```
def node_order(self) -> builtins.list[NodeIndex]:
```

```
Node order from traversal.
```

`covers`

Kind: `Method`

```
def covers(self, subgraph: Subgraph) -> Subgraph:
```

```
Covers a subgraph with the traversal tree.
```

`subgraph`

Kind: `Parameter`

```
subgraph: Subgraph
```

`iter_hedges`

Kind: `Method`

```
def iter_hedges(self) -> builtins.list[tuple[Hedge, builtins.str,
typing.Optional[Hedge]]]:
```

```
Iterate hedges as (hedge, kind, root_hedge).
```

Exhaustive reference · Source

Canonical package changelog

The following release history is imported as typed Markdown from `crates/linnet/CHANGELOG.md`.

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

[Unreleased]

0.17.0 - 2026-01-28

Other

- force-directed layout
- make the powerset iterator properly size the subset
- upgrade to all spanning *forests* generation
- Refactor PowersetIterator to support generic ID type
- double ended iterator for subsets
- remove unused crates

0.16.1 - 2026-01-13

Other

- Quote string values in graph and parser outputs
- revert edge, node and hedge index order to be direct (not based on sort)

0.16.0 - 2026-01-08

Other

- Add methods for identifying bridges and non-bridges in subgraphs
- Make trace_unfold implementable outside of linnet

0.15.1 - 2026-01-05

Fixed

- fixed ordering problem in merge function and make ordering based on sorting not on direct map

Other

- update quote stripping
- Add agent context files and update project tooling
- Add graph algorithms module with topological sort and transitive ops

0.15.0 - 2025-12-08

Other

- Refactor Figment handling and improve data merging logic
- *(clinnet)* release v0.1.8

0.14.1 - 2025-12-03

Other

- Add parallel processing and directional constraints
- update test

- only swap one edge when connecting identities

0.14.0 - 2025-11-28

Other
- change output name and make it short, and add directional groups

0.13.3 - 2025-11-24

Other
- update symbolica

0.13.2 - 2025-11-12

Other
- update to symbolica 0.20
- better templates
- clinnet for nested dots to pdf
- Add Figment integration for configuration handling

0.13.1 - 2025-11-05

Other
- remove prints

0.12.0 - 2025-08-18

Other
- all tests pass
- clippy fixes
- write global data outside of graph[] scope
- Add ID=ID statements to global data when parsing
- newline after nodes
- dot serialize with name
- write graph name
- correctly parse orientations
- make iter_edges iterate with edge_id order not involution order

0.11.0 - 2025-07-29

Other
- serialize global data

0.10.0 - 2025-07-29

Other
- Add check graph function and extracting by nodes
- Add is_empty to Swap trait and update implementations

0.9.1 - 2025-07-28

Other
- impl asref for newtypes

0.9.0 - 2025-07-27

Other

- Allow specifying a subgraph in the DOT format.
- Rewrite DOT output to include node, hedge and edge indices and fix bidirectional serialization
- DotGraph has concrete data. Parser now supports global edge and vertex data. Can set edge, vertex and hedge index
- add multi-graph parsing and global graph data

0.8.0 - 2025-07-14

Other

- clippy fix
- Rename HedgeVec to EdgeVec and introduce HedgeVec
- Add H parameter to HedgeGraph in iter_edges and orientation
- add H to SignedCycle
- rename as_ref to to_ref
- Add hedge_data to hedge_graph
- add perm by key
- update symbolica
- Fix tree traversal and modify ForestNodeStore API to use Option

0.7.1 - 2025-05-24

Other

- Fix crown computation by inverting subgraph inclusion check and add inclusion for HedgePair

0.7.0 - 2025-05-23

Other

- unify iterator signature and remove redundant and misleading functions
- Add zenodo doi

0.6.3 - 2025-05-23

Other

- improve README

0.6.2 - 2025-05-22

Other

- remove useless check and set
- Correctly forget identification history and adds subgraph contraction

0.6.1 - 2025-05-22

Other

- Merge pull request #11 from alphal00p/jules_wip_3253120278989361716

0.6.0 - 2025-05-22

Fixed

- fix all tests and update symbolica

- fixes forget_identification_history, If you delete a subgraph you need

Other

- Add crown functions and implement spanning tree enumeration algo based on graph contraction
- PartialEq and Eq for hedgevec
- from_raw for hedgevec

0.5.2 - 2025-05-09

Fixed

- fix involution extraction

Other

- Add Permutation iterators, new graph extract test

0.5.1 - 2025-05-05

Other

- Clean up debug prints and add cycle test

0.5.0 - 2025-05-05

Fixed

- fixes all around

Other

- boundary->crown
- hairs->boundary

0.4.0 - 2025-04-30

Added

- feature flag bincode and serde

0.3.0 - 2025-04-30

Fixed

- fixed extraction for childvec forest, all tests pass and fix warnings

Other

- adds extract to SmartHedgeVec and involution
- maps now take a FnMut
- derive more (Decode and Encode)

0.2.3 - 2025-04-08

Other

- unpin serde

0.2.2 - 2025-04-01

Fixed

- fix test snapshots

Other

- raw image

0.2.1 - 2025-04-01

Other

- Update README.md, logo is a github asset
- put crates badge

0.2.0 - 2025-04-01

Fixed

- fix cetz orientation

Other

- Create release-plz.yml

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/linnet/latest/`
- **Source revision:** `b200a8f5931914d9832f64b190779583454be49f`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python_api, python_abi, python_stubgen`)
- `linnet/linnet` — `0.17.0` (`features: nodestore-vec, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3-py310`)
- `spenso/spenso` — `0.6.0` (`features: shadowing`)
- `spenso/spenso-macros` — `0.3.2` (`features: shadowing`)
- `spenso/spenso-hep-lib` — `0.1.7` (`no optional features`)
- `spenso/spynso3` — `0.1.2` (`features: python_stubgen`)
- `idenso/idenso` — `0.3.0` (`features: bincode, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python_stubgen`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: symbolica_community_module, python_stubgen`)